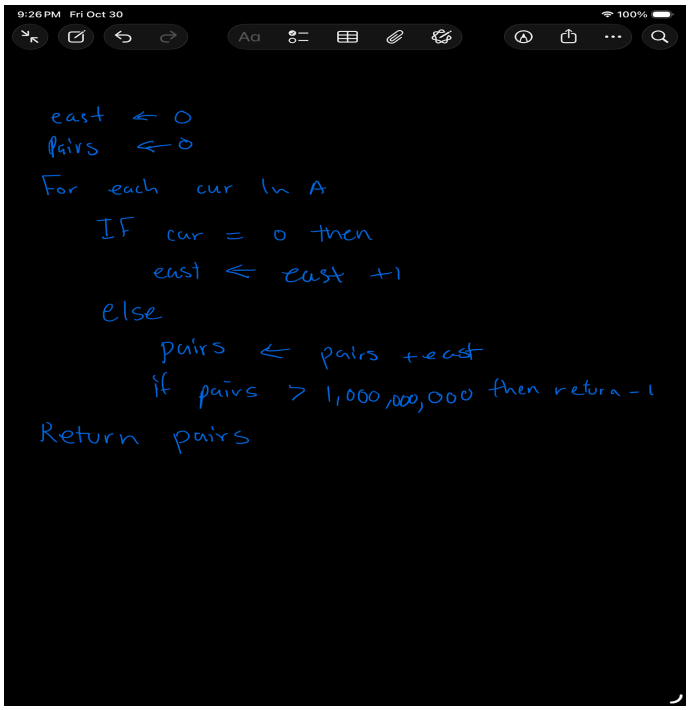
Assignment 1 - codility problems: passing cars
Michael Chang 10/30/2026



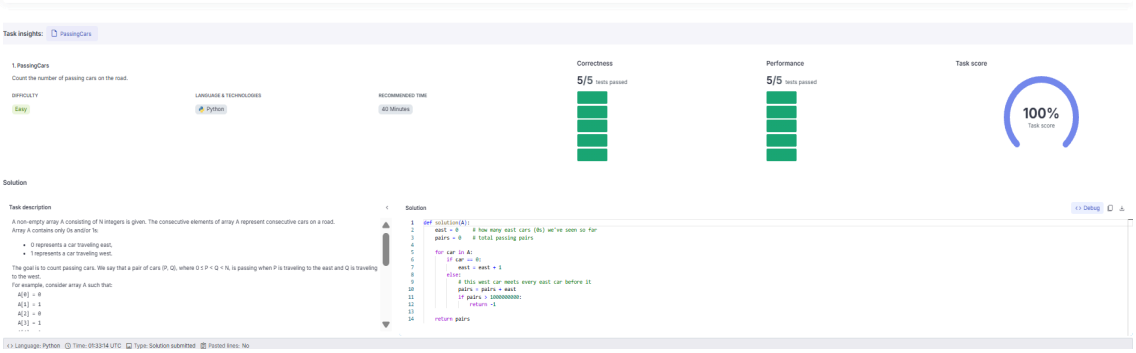

Written code

```
def solution(A):
    east = 0     # how many east cars (0s) we've seen so far
    pairs = 0    # total passing pairs

    for car in A:
        if car == 0:
            east = east + 1
        else:
            # this west car meets every east car before it
            pairs = pairs + east
            if pairs > 1000000000:
                return -1

    return pairs
```

Codility Correctness Tests

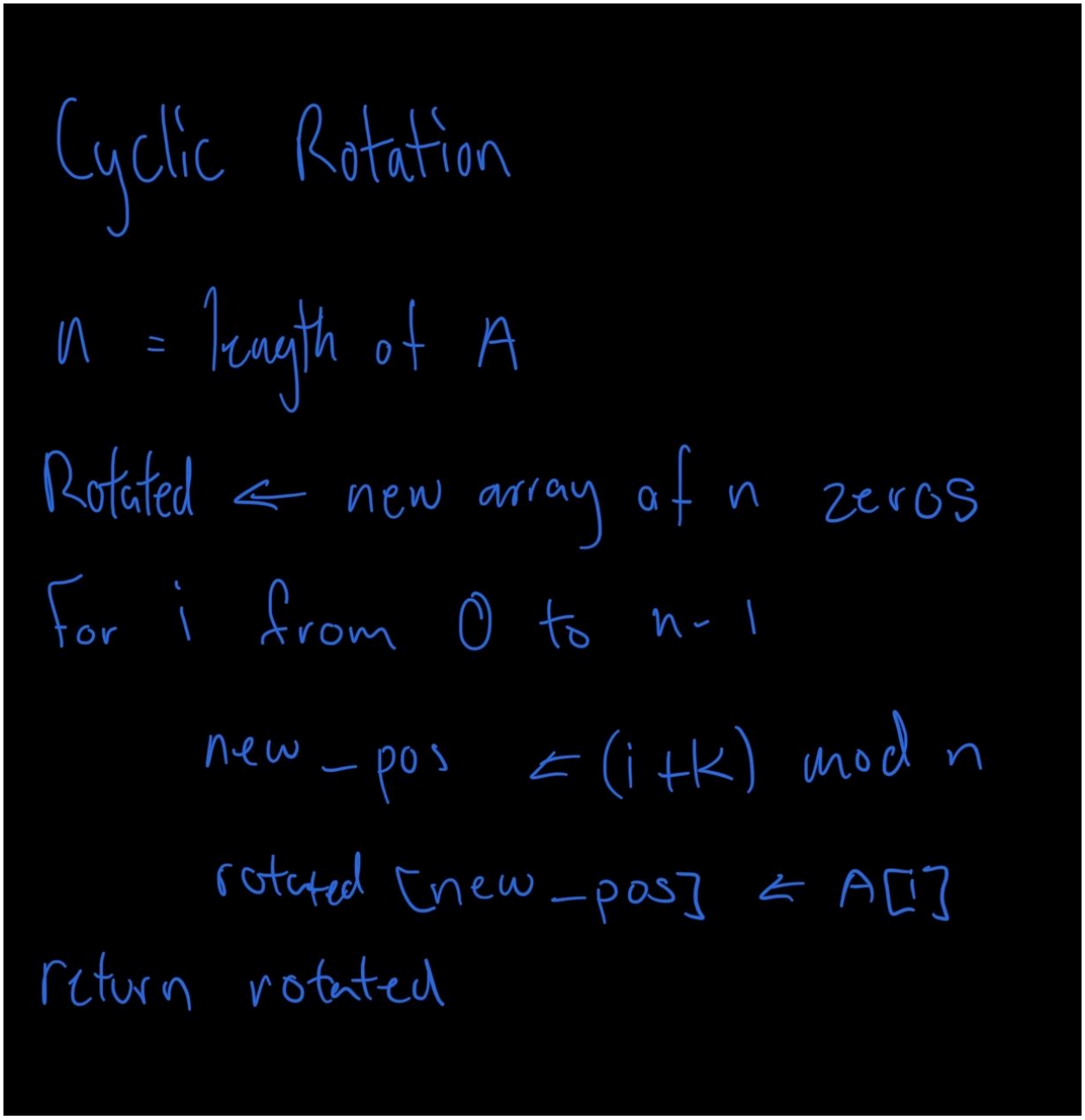



Citations:
Anthropic. (2026). Claude Opus 5.5 [Large language model]. https://claude.ai. Used to select the problem, explain the approach, and draft the pseudocode and Python solution.

Codility. (n.d.). PassingCars [Programming lesson]. https://app.codility.com/programmers/lessons/5-prefix_sums/passing_cars/

Links:
AI chat log: https://claude.ai/share/6870113c-9609-44a7-b740-7c2c3cce9b17
GitHub repository: https://github.com/m-chang24/codility-assignment-1

CyclicRotation



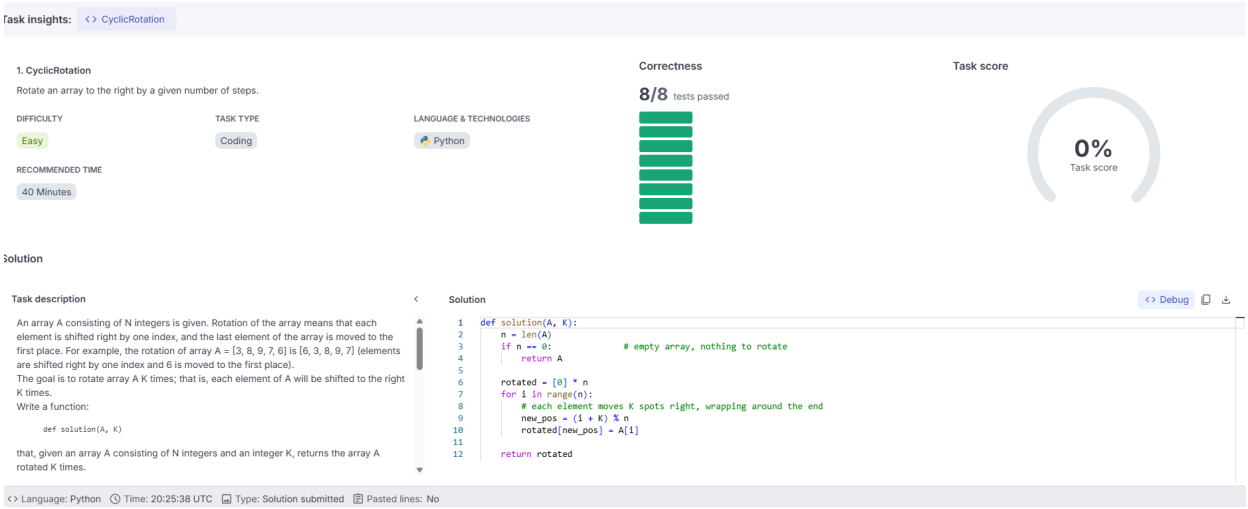

Written code:

```
def solution(A, K):
    n = len(A)
    if n == 0:              # empty array, nothing to rotate
        return A

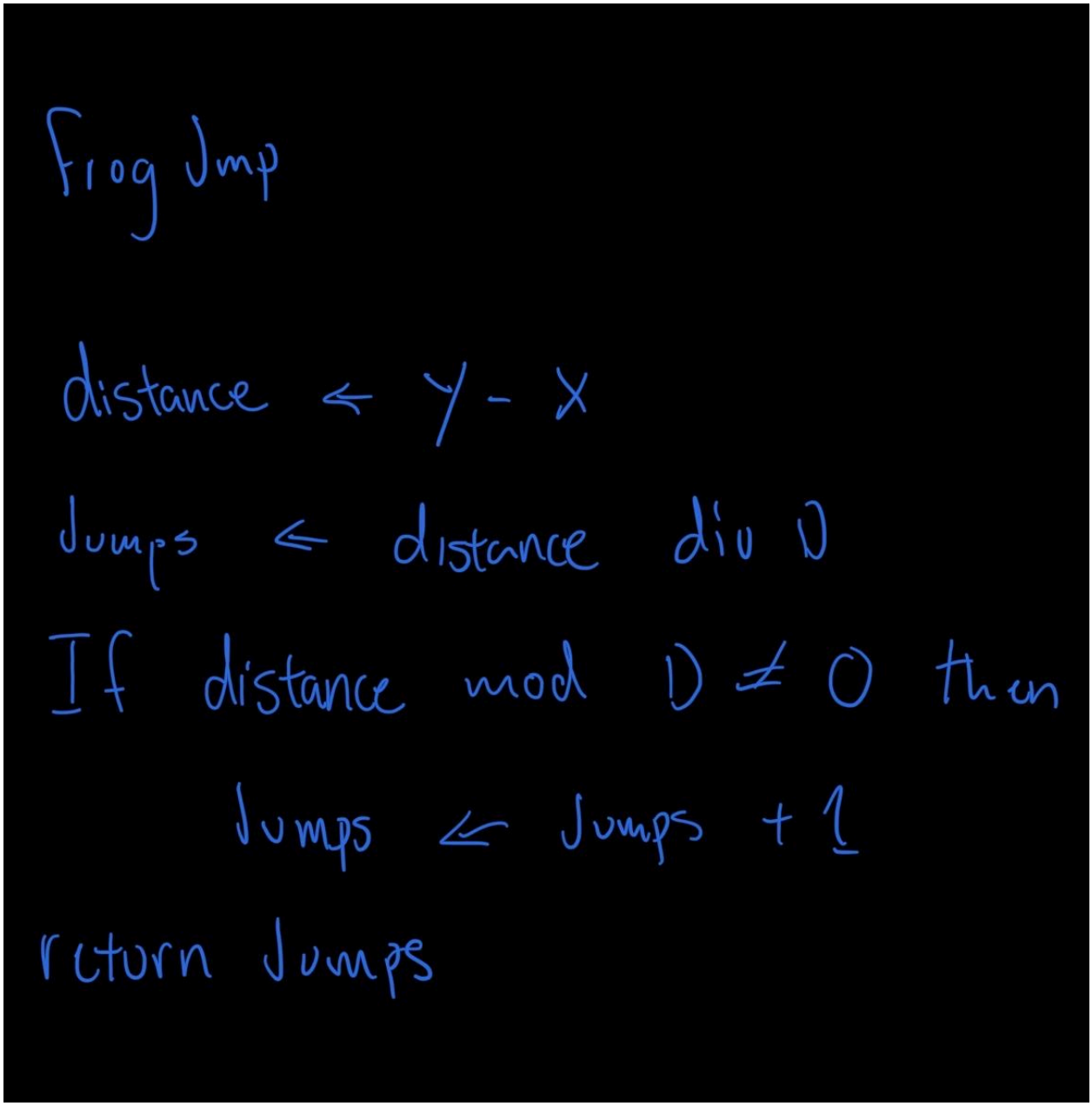
    rotated = [0] * n
    for i in range(n):
        # each element moves K spots right, wrapping around the end
```

```
    new_pos = (i + K) % n
    rotated[new_pos] = A[i]

return rotated
```



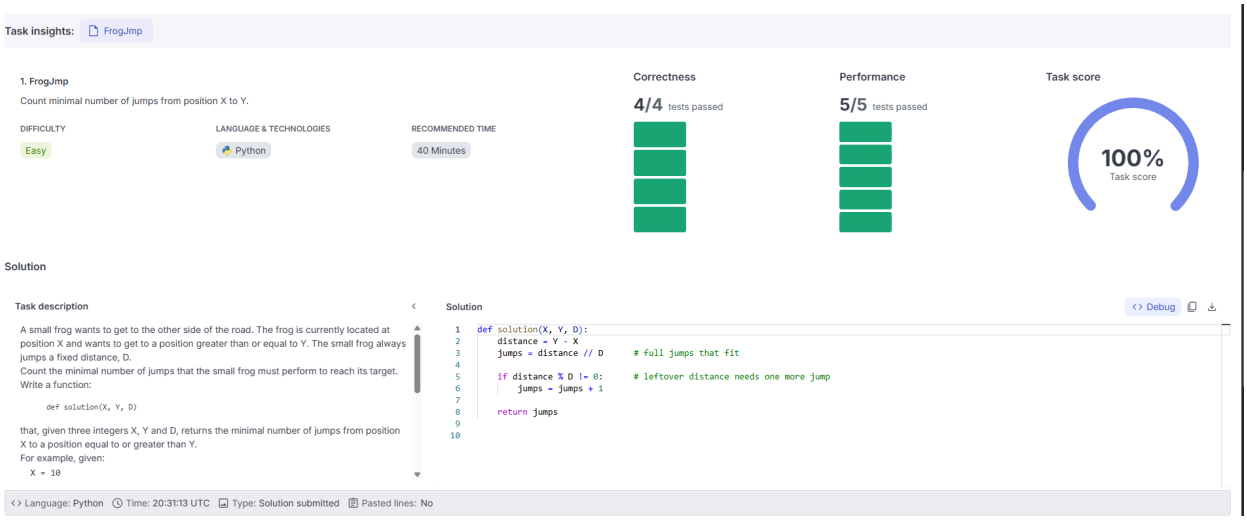

Frogjmp



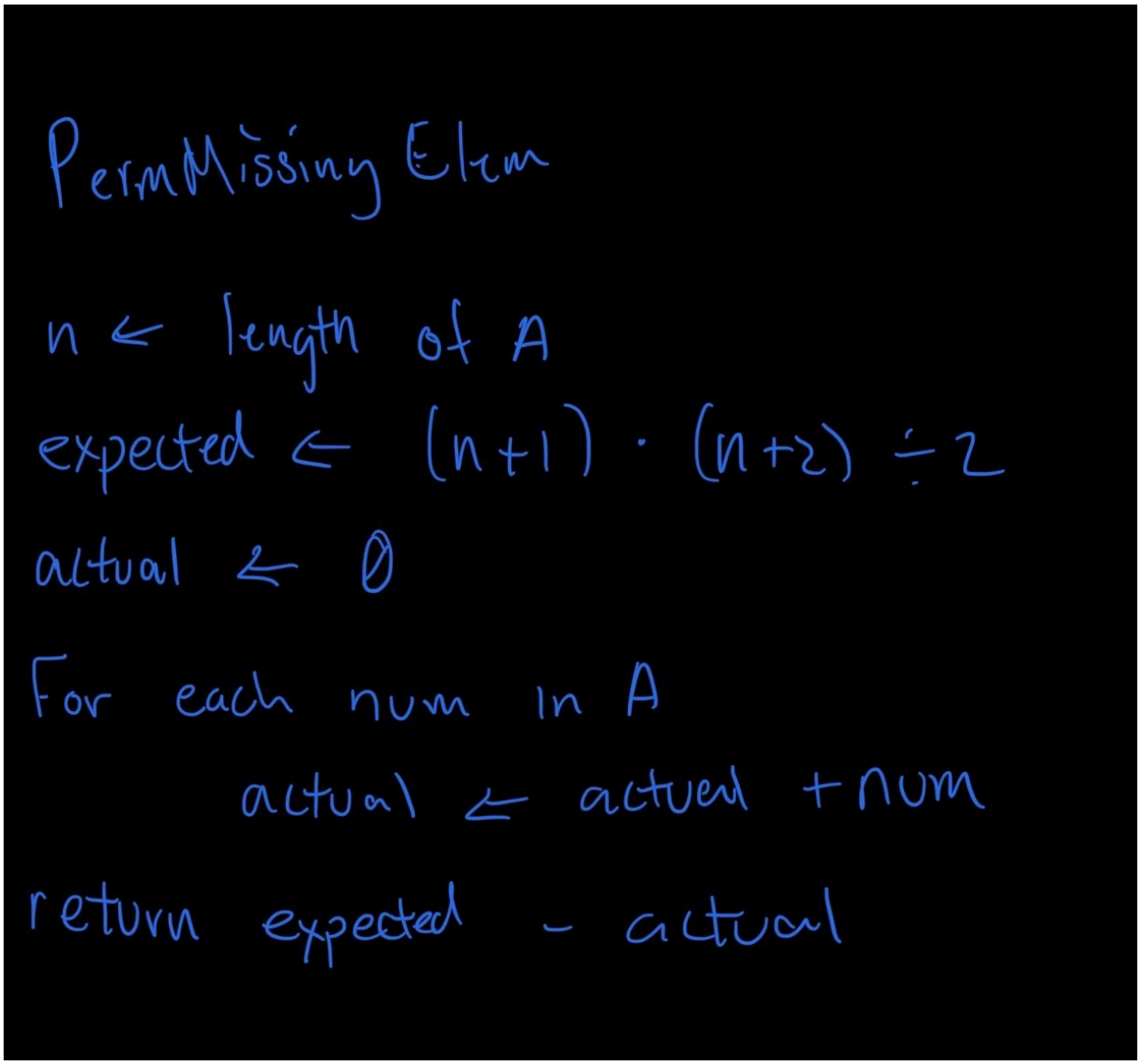

```
def solution(X, Y, D):
    distance = Y - X
    jumps = distance // D      # full jumps that fit

    if distance % D != 0:      # leftover distance needs one more jump
        jumps = jumps + 1

    return jumps
```



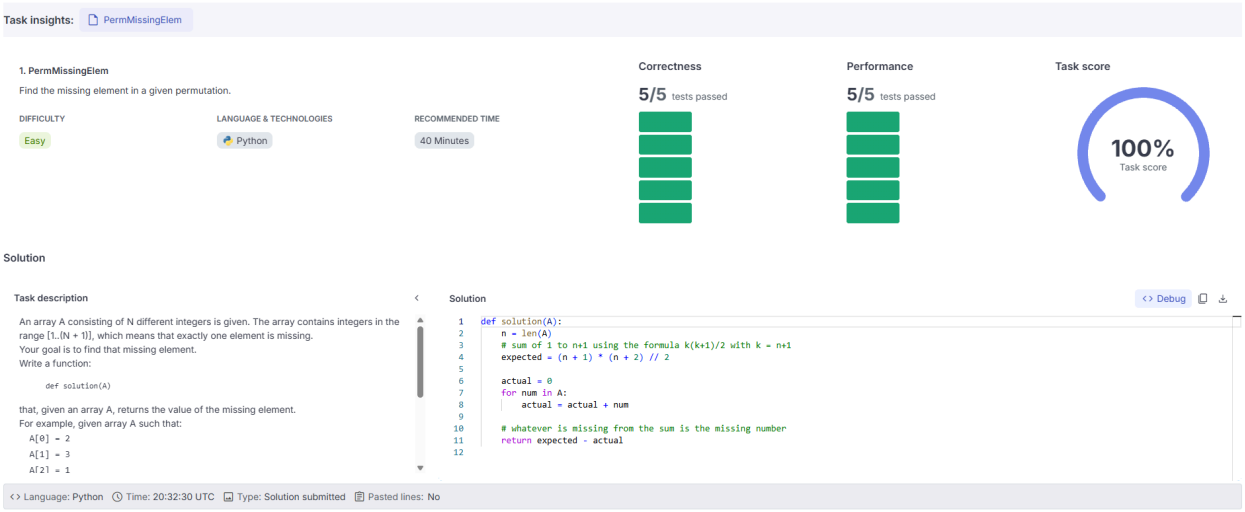

PermMissingElem



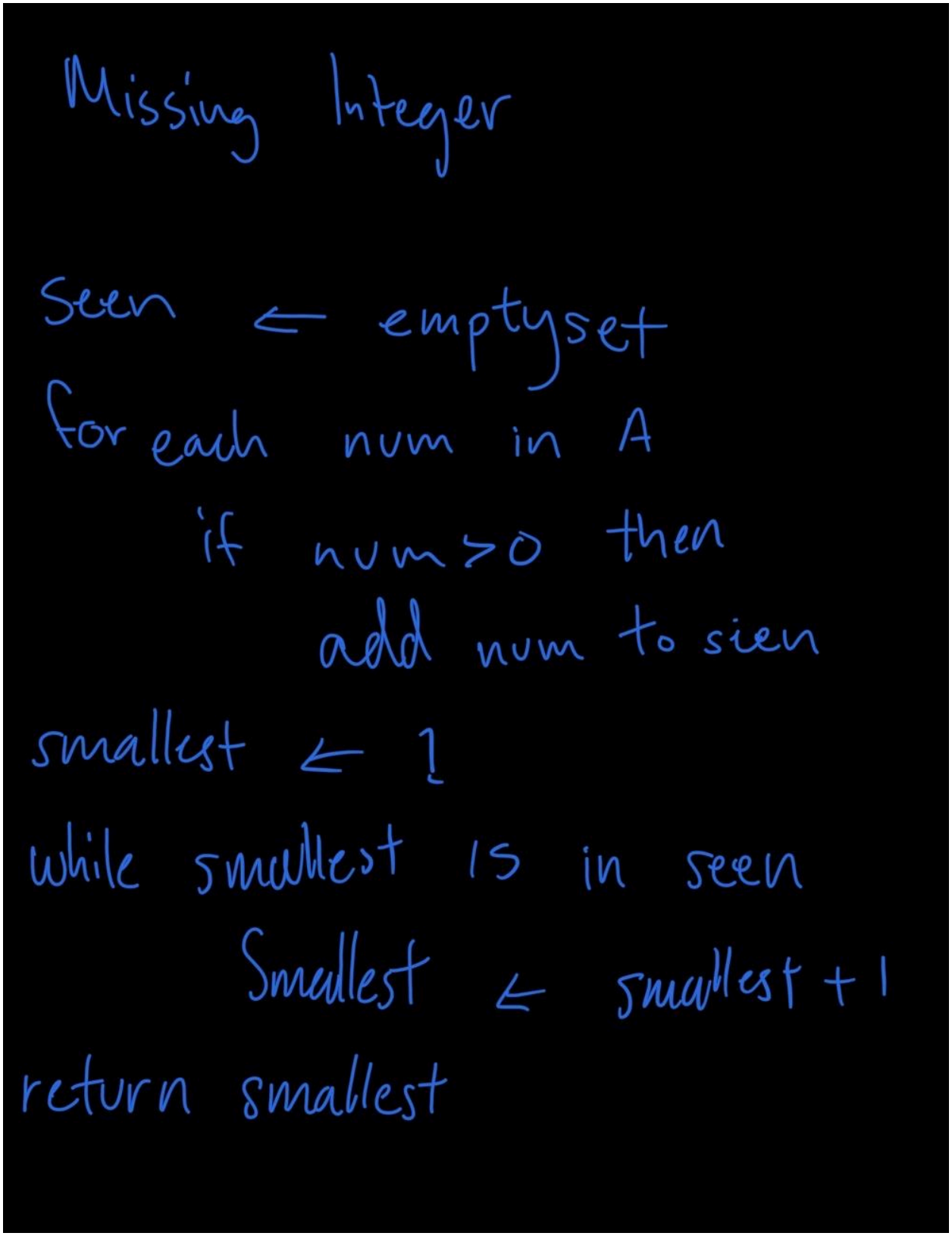

```
def solution(A):
    n = len(A)
    # sum of 1 to n+1 using the formula k(k+1)/2 with k = n+1
    expected = (n + 1) * (n + 2) // 2

    actual = 0
    for num in A:
        actual = actual + num

    # whatever is missing from the sum is the missing number
    return expected - actual
```




# MissingInteger

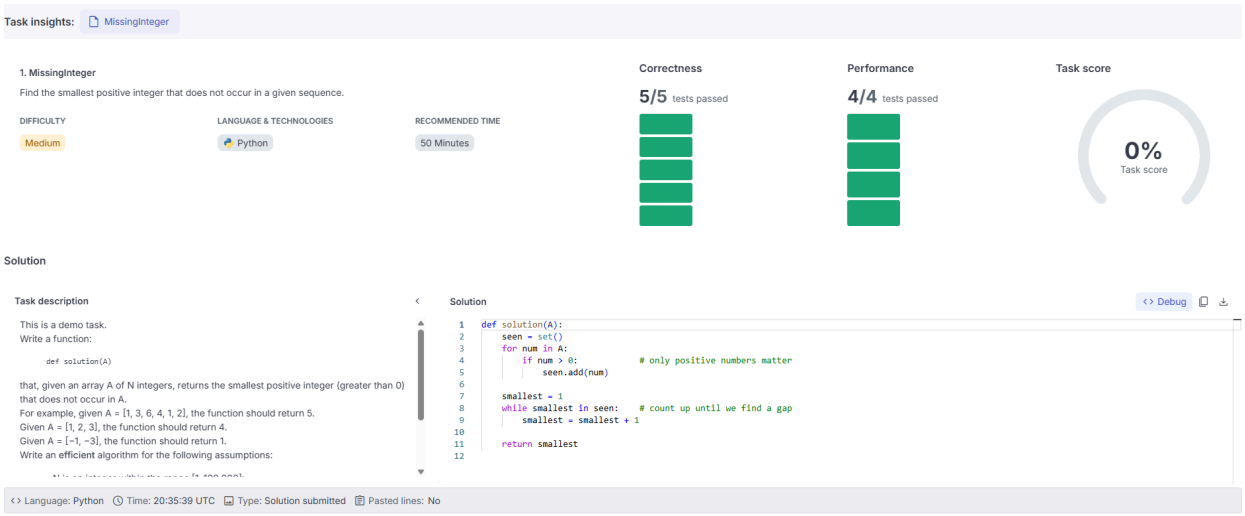



```
def solution(A):
    seen = set()
```

```
    for num in A:
        if num > 0:            # only positive numbers matter
            seen.add(num)

    smallest = 1
    while smallest in seen:    # count up until we find a gap
        smallest = smallest + 1

    return smallest
```




**Citations:**


Anthropic. (2026). *Claude Opus 5.5* [Large language model]. https://claude.ai. Used to select the problems, explain the approaches, debug and finalize solutions.

AI chat log:
https://claude.ai/share/6870113c-9609-44a7-b740-7c2c3cce9b17
https://claude.ai/share/67ca09b9-90e1-42e1-87f1-c2fa7d857953

Codility. (n.d.). *CyclicRotation* [Programming lesson]. https://app.codility.com/programmers/lessons/2-arrays/cyclic_rotation/

Codility. (n.d.). *FrogJmp* [Programming lesson]. https://app.codility.com/programmers/lessons/3-time_complexity/frog_jmp/

Codility. (n.d.). *MissingInteger* [Programming lesson]. https://app.codility.com/programmers/lessons/4-counting_elements/missing_integer/

Codility. (n.d.). *PassingCars* [Programming lesson].
https://app.codility.com/programmers/lessons/5-prefix_sums/passing_cars/

Codility. (n.d.). *PermMissingElem* [Programming lesson].
https://app.codility.com/programmers/lessons/3-time_complexity/perm_missing_elem